CLI Command Restructure

Date: 2026-03-17
Status: Draft

Problem

Outport's CLI commands evolved organically over 4 days of development:

1. First came port allocation: `register` / `unregister`
2. Then `.env` fencing enabled clean reversal: `apply` / `unapply`
3. Then the daemon (DNS, reverse proxy, HTTPS): `setup` / `teardown`, `up` / `down`

The result is a flat list of 14 commands where project-scoped and system-scoped operations are mixed together. A new user seeing `up`, `down`, `apply`, `unapply`, `setup`, and `teardown` has no clear mental model of what to run or when.

The daemon and `.test` domains are now the default experience, not an optional add-on. The CLI should reflect that.

Design Principles

- **Project commands are top-level.** The most common thing a developer does is work in a project directory. Those commands should be the shortest and most prominent.
- **System commands are namespaced.** Machine-wide operations (daemon, DNS, CA, registry) live under `outport system`. They're rarer and more consequential.
- **Docker Compose mental model.** `outport up` in a directory means "bring this project into outport." `outport down` means "take it out." This matches the muscle memory most developers have.
- **Smart defaults over ceremony.** `outport system start` auto-detects whether first-time setup is needed and does it. One command, not two.
- **No backward compatibility.** This is a pre-alpha tool with one user. Optimize for the best UX.

Command Structure

Project Commands (top-level)

Run in a project directory, operate on that project's configuration and registry entry.

Command	Description	Replaces	Flags
`outport init`	Create `.outport.yml` template	*(unchanged)*	`--json`
`outport up`	Register project, allocate ports, write `.env`	`apply`	`--force`, `--json`
`outport down`	Clean `.env` blocks, remove from registry	`unapply`	`--json`
`outport ports`	Show allocated ports for current project	*(unchanged)*	`--check`, `--derived`, `--json`
`outport open [service]`	Open HTTP services in the browser	*(unchanged)*	`--json`
`outport rename <old> <new>`	Rename a project instance	*(unchanged)*	`--json`
`outport promote`	Promote current instance to main	*(unchanged)*	`--json`

System Commands (`outport system`)

Operate on the machine-wide outport installation: daemon, DNS resolver, CA, and global registry.

Command	Description	Replaces	Notes
`outport system start`	Start the daemon (auto-setup on first run)	`setup` + `up`	Idempotent. First run: writes plist, installs DNS resolver (sudo), generates CA, trusts CA (keychain prompt), loads agent.

Command	Description	Replaces	Notes
			Subsequent runs: loads agent if not running. `--json`
`outport system stop`	Stop the daemon	`down`	Unloads LaunchAgent. Non-destructive. `--json`
`outport system restart`	Restart the daemon	*(new)*	Stop + start. Re-writes plist (re-resolves binary path) but does NOT re-run full setup (no sudo, no CA). Errors if setup hasn't been done. `--json`
`outport system status`	Show all registered projects	`status`	Shows all projects system-wide. `--check`, `--derived`, `--json` flags. Shows stale markers but does NOT prompt for removal — use `system gc` instead.
`outport system gc`	Remove stale registry entries	`gc`	Removes entries where directory or config no longer exists. `--json`
`outport system uninstall`	Remove DNS, daemon, CA, and certs	`teardown`	Full cleanup: unload agent, remove plist, remove `/etc/resolver/test`, untrust and delete CA, clear cert cache. `--json`

Hidden Commands

Command	Description	Notes
`outport daemon`	Run DNS/proxy daemon directly	Invoked by launchd, not by users. Unchanged.

Smart Behavior

`outport system start` (auto-setup)

```
if platform.IsSetup():
    if daemon already running:
        print "Already running."
        return
    load agent
else:
    check ports 80/443 not in use
    find outport binary in PATH
    write LaunchAgent plist
    write /etc/resolver/test (sudo prompt)
    generate CA if not exists
    trust CA in keychain (GUI prompt)
    load agent
    print "Done! *.test domains are now routing with HTTPS."
```

`outport up` (daemon hint)

When the daemon is not running, `outport up` prints a non-blocking hint after its normal output:

```
Hint: The outport daemon is not running. Run `outport system start` to enable .test dom
```

It does NOT auto-start the system because that involves sudo and keychain prompts that shouldn't surprise the user. Port allocation and `.env` writing still work without the daemon.

`outport system restart`

Re-writes the LaunchAgent plist (re-resolves the outport binary path via `exec.LookPath`) then does stop + start. This ensures that after `brew upgrade outport`, the daemon picks up the new binary. Does NOT re-run the full setup flow (no sudo for DNS, no CA generation). If setup hasn't been done, it errors with: "Outport is not set up. Run `outport system start` to install."

Removed Commands

These commands are deleted entirely, not aliased:

- `apply` → replaced by `up`
- `unapply` → replaced by `down`
- `setup` → replaced by `system start`
- `teardown` → replaced by `system uninstall`
- `up` (top-level) → replaced by `system start`
- `down` (top-level) → replaced by `system stop`
- `status` (top-level) → replaced by `system status`
- `gc` (top-level) → replaced by `system gc`

User Journey

First-time setup

```
brew install outport
outport system start          # installs DNS, CA, starts daemon
```

Adding a project

```
cd ~/src/myapp
outport init                  # creates .outport.yml
vim .outport.yml              # configure services
outport up                    # allocate ports, write .env — done
outport open                  # open in browser
```

Day-to-day

```
outport ports                 # what ports do I have?
outport open web              # open a service
outport system status         # what's registered across all projects?
```

Worktrees / multiple instances

```
cd ~/src/myapp-feature
outport up                    # auto-registers as new instance
outport rename bxcf feature   # give it a readable name
outport promote               # make this the main instance
```

Cleanup

```
outport down                  # remove this project from outport
outport system gc             # clean stale entries
outport system uninstall      # remove everything (nuclear option)
```

Upgrading outport

```
brew upgrade outport
outport system restart        # bounce the daemon to pick up new binary
```

Help Output

```
Dev port manager for multi-project development

Usage:
  outport [command]

Project Commands:
  init              Create .outport.yml for this project
  up                Bring this project into outport
  down              Remove this project from outport
  ports             Show ports for the current project
  open              Open HTTP services in the browser
  rename            Rename an instance of the current project
  promote           Promote the current instance to main

System Commands:
  system start      Start the outport system
  system stop       Stop the outport system
  system restart    Restart the outport system
  system status     Show all registered projects
  system gc         Clean stale entries from the registry
  system uninstall  Remove outport system components

Flags:
  -h, --help     help for outport
      --json     output as JSON
  -v, --version  version for outport

Use "outport [command] --help" for more information about a command.
```

Implementation Scope

Files to modify

Command files (restructure):

- `cmd/apply.go` → rename to `cmd/up.go`, change Use/Short/Long, drop `a` alias
- `cmd/unapply.go` → rename to `cmd/down.go`, change Use/Short/Long

- `cmd/updown.go` → delete (replaced by system subcommands)
- `cmd/setup.go` → refactor into system subcommands
- `cmd/status.go` → move under system subcommand, remove interactive stale-entry prompt (defer to `system gc`)
- `cmd/gc.go` → move under system subcommand
- `cmd/root.go` → add `system` parent command with Cobra `AddGroup()` for grouped help output, re-register subcommands
- New: `cmd/system.go` — parent command + start/stop/restart/uninstall subcommands

User-facing strings referencing old command names:

- `cmd/init.go` — references to `outport apply` → `outport up`, `outport setup` → `outport system start`
- `cmd/ports.go` — "Run 'outport apply' first" → "Run 'outport up' first"
- `cmd/open.go` — "Run 'outport apply' first" → "Run 'outport up' first"
- `internal/daemon/proxy.go` — HTML error page references `outport apply` → `outport up`
- `internal/platform/other.go` — references `outport setup` → `outport system start`

Tests:

- `cmd/cmdutil_test.go` — hardcoded command name lists must be updated. `TestAllCommandsHaveArgsValidation` iterates `rootCmd.Commands()` — needs to recurse into `systemCmd.Commands()` for subcommands.
- All test files referencing old command names in assertions or setup.

Files to update after

- `README.md` — command list and getting started
- `docs/` — guide and reference pages
- `CLAUDE.md` — architecture section, command descriptions

No changes needed

- `internal/` packages (except `daemon/proxy.go` and `platform/other.go` noted above)
- `cmd/rename.go`, `cmd/promote.go` — unchanged
- `cmd/daemon.go` — hidden, unchanged
- `cmd/context.go` — unchanged